

#	Description	Level
13.3.1	Verify that a secrets management solution, such as a key vault, is used to securely create, store, control access to, and destroy backend secrets. These could include passwords, key material, integrations with databases and third-party systems, keys and seeds for time-based tokens, other internal secrets, and API keys. Secrets must not be included in application source code or included in build artifacts. For an L3 application, this must involve a hardware-backed solution such as an HSM.	2
13.3.2	Verify that access to secret assets adheres to the principle of least privilege.	2
13.3.3	Verify that all cryptographic operations are performed using an isolated security module (such as a vault or hardware security module) to securely manage and protect key material from exposure outside of the security module.	3
13.3.4	Verify that secrets are configured to expire and be rotated based on the application's documentation.	3

V13.4 Unintended Information Leakage

Production configurations should be hardened to avoid disclosing unnecessary data. Many of these issues are rarely rated as significant risks but are often chained with other vulnerabilities. If these issues are not present by default, it raises the bar for attacking an application.

For example, hiding the version of server-side components does not eliminate the need to patch all components, and disabling folder listing does not remove the need to use authorization controls or keep files away from the public folder, but it raises the bar.

#	Description	Level
13.4.1	Verify that the application is deployed either without any source control metadata, including the .git or .svn folders, or in a way that these folders are inaccessible both externally and to the application itself.	1
13.4.2	Verify that debug modes are disabled for all components in production environments to prevent exposure of debugging features and information leakage.	2
13.4.3	Verify that web servers do not expose directory listings to clients unless explicitly intended.	2
13.4.4	Verify that using the HTTP TRACE method is not supported in production environments, to avoid potential information leakage.	2

#	Description	Level
13.4.5	Verify that documentation (such as for internal APIs) and monitoring endpoints are not exposed unless explicitly intended.	2
13.4.6	Verify that the application does not expose detailed version information of backend components.	3
13.4.7	Verify that the web tier is configured to only serve files with specific file extensions to prevent unintentional information, configuration, and source code leakage.	3

References

For more information, see also:

- OWASP Web Security Testing Guide: Configuration and Deployment Management Testing

V14 Data Protection

Control Objective

Applications cannot account for all usage patterns and user behaviors, and should therefore implement controls to limit unauthorized access to sensitive data on client devices.

This chapter includes requirements related to defining what data needs to be protected, how it should be protected, and specific mechanisms to implement or pitfalls to avoid.

Another consideration for data protection is bulk extraction, modification, or excessive usage. Each system's requirements are likely to be very different, so determining what is "abnormal" must consider the threat model and business risk. From an ASVS perspective, detecting these issues is handled in the "Security Logging and Error Handling" chapter, and setting limits is handled in the "Validation and Business Logic" chapter.

V14.1 Data Protection Documentation

A key prerequisite for being able to protect data is to categorize what data should be considered sensitive. There are likely to be several different levels of sensitivity, and for each level, the controls required to protect data at that level will be different.

There are various privacy regulations and laws that affect how applications must approach the storage, use, and transmission of sensitive personal information. This section no longer tries to duplicate

these types of data protection or privacy legislation, but rather focuses on key technical considerations for protecting sensitive data. Please consult local laws and regulations, and consult a qualified privacy specialist or lawyer as required.

#	Description	Level
14.1.1	Verify that all sensitive data created and processed by the application has been identified and classified into protection levels. This includes data that is only encoded and therefore easily decoded, such as Base64 strings or the plaintext payload inside a JWT. Protection levels need to take into account any data protection and privacy regulations and standards which the application is required to comply with.	2
14.1.2	Verify that all sensitive data protection levels have a documented set of protection requirements. This must include (but not be limited to) requirements related to general encryption, integrity verification, retention, how the data is to be logged, access controls around sensitive data in logs, database-level encryption, privacy and privacy-enhancing technologies to be used, and other confidentiality requirements.	2

V14.2 General Data Protection

This section contains various practical requirements related to the protection of data. Most are specific to particular issues such as unintended data leakage, but there is also a general requirement to implement protection controls based on the protection level required for each data item.

#	Description	Level
14.2.1	Verify that sensitive data is only sent to the server in the HTTP message body or header fields, and that the URL and query string do not contain sensitive information, such as an API key or session token.	1
14.2.2	Verify that the application prevents sensitive data from being cached in server components, such as load balancers and application caches, or ensures that the data is securely purged after use.	2
14.2.3	Verify that defined sensitive data is not sent to untrusted parties (e.g., user trackers) to prevent unwanted collection of data outside of the application's control.	2

#	Description	Level
14.2.4	Verify that controls around sensitive data related to encryption, integrity verification, retention, how the data is to be logged, access controls around sensitive data in logs, privacy and privacy-enhancing technologies, are implemented as defined in the documentation for the specific data's protection level.	2
14.2.5	Verify that caching mechanisms are configured to only cache responses which have the expected content type for that resource and do not contain sensitive, dynamic content. The web server should return a 404 or 302 response when a non-existent file is accessed rather than returning a different, valid file. This should prevent Web Cache Deception attacks.	3
14.2.6	Verify that the application only returns the minimum required sensitive data for the application's functionality. For example, only returning some of the digits of a credit card number and not the full number. If the complete data is required, it should be masked in the user interface unless the user specifically views it.	3
14.2.7	Verify that sensitive information is subject to data retention classification, ensuring that outdated or unnecessary data is deleted automatically, on a defined schedule, or as the situation requires.	3
14.2.8	Verify that sensitive information is removed from the metadata of user-submitted files unless storage is consented to by the user.	3

V14.3 Client-side Data Protection

This section contains requirements preventing data from leaking in specific ways at the client or user agent side of an application.

#	Description	Level
14.3.1	Verify that authenticated data is cleared from client storage, such as the browser DOM, after the client or session is terminated. The 'Clear-Site-Data' HTTP response header field may be able to help with this but the client-side should also be able to clear up if the server connection is not available when the session is terminated.	1
14.3.2	Verify that the application sets sufficient anti-caching HTTP response header fields (i.e., Cache-Control: no-store) so that sensitive data is not cached in browsers.	2

#	Description	Level
14.3.3	Verify that data stored in browser storage (such as localStorage, sessionStorage, IndexedDB, or cookies) does not contain sensitive data, with the exception of session tokens.	2

References

For more information, see also:

- Consider using the Security Headers website to check security and anti-caching header fields
- Documentation about anti-caching headers by Mozilla
- OWASP Secure Headers project
- OWASP Privacy Risks Project
- OWASP User Privacy Protection Cheat Sheet
- Australian Privacy Principle 11 - Security of personal information
- European Union General Data Protection Regulation (GDPR) overview
- European Union Data Protection Supervisor - Internet Privacy Engineering Network
- Information on the “Clear-Site-Data” header
- White paper on Web Cache Deception

V15 Secure Coding and Architecture

Control Objective

Many ASVS requirements either relate to a particular area of security, such as authentication or authorization, or pertain to a particular type of application functionality, such as logging or file handling.

This chapter provides general security requirements to consider when designing and developing applications. These requirements focus not only on clean architecture and code quality but also on specific architecture and coding practices necessary for application security.

V15.1 Secure Coding and Architecture Documentation

Many requirements for establishing a secure and defensible architecture depend on clear documentation of decisions made regarding the implementation of specific security controls and the components used within the application.

This section outlines the documentation requirements, including identifying components considered to contain “dangerous functionality” or to be “risky components.”

A component with “dangerous functionality” may be an internally developed or third-party component that performs operations such as deserialization of untrusted data, raw file or binary data parsing, dynamic code execution, or direct memory manipulation. Vulnerabilities in these types of operations pose a high risk of compromising the application and potentially exposing its underlying infrastructure.

A “risky component” is a 3rd party library (i.e., not internally developed) with missing or poorly implemented security controls around its development processes or functionality. Examples include components that are poorly maintained, unsupported, at the end-of-life stage, or have a history of significant vulnerabilities.

This section also emphasizes the importance of defining appropriate timeframes for addressing vulnerabilities in third-party components.

#	Description	Level
15.1.1	Verify that application documentation defines risk based remediation time frames for 3rd party component versions with vulnerabilities and for updating libraries in general, to minimize the risk from these components.	1
15.1.2	Verify that an inventory catalog, such as software bill of materials (SBOM), is maintained of all third-party libraries in use, including verifying that components come from pre-defined, trusted, and continually maintained repositories.	2
15.1.3	Verify that the application documentation identifies functionality which is time-consuming or resource-demanding. This must include how to prevent a loss of availability due to overusing this functionality and how to avoid a situation where building a response takes longer than the consumer’s timeout. Potential defenses may include asynchronous processing, using queues, and limiting parallel processes per user and per application.	2
15.1.4	Verify that application documentation highlights third-party libraries which are considered to be “risky components”.	3
15.1.5	Verify that application documentation highlights parts of the application where “dangerous functionality” is being used.	3

V15.2 Security Architecture and Dependencies

This section includes requirements for handling risky, outdated, or insecure dependencies and components through dependency management.

It also includes using architectural-level techniques such as sandboxing, encapsulation, containerization, and network isolation to reduce the impact of using “dangerous operations” or “risky compo-

nents”(as defined in the previous section) and prevent loss of availability due to overusing resource-demanding functionality.

#	Description	Level
15.2.1	Verify that the application only contains components which have not breached the documented update and remediation time frames.	1
15.2.2	Verify that the application has implemented defenses against loss of availability due to functionality which is time-consuming or resource-demanding, based on the documented security decisions and strategies for this.	2
15.2.3	Verify that the production environment only includes functionality that is required for the application to function, and does not expose extraneous functionality such as test code, sample snippets, and development functionality.	2
15.2.4	Verify that third-party components and all of their transitive dependencies are included from the expected repository, whether internally owned or an external source, and that there is no risk of a dependency confusion attack.	3
15.2.5	Verify that the application implements additional protections around parts of the application which are documented as containing “dangerous functionality” or using third-party libraries considered to be “risky components”. This could include techniques such as sandboxing, encapsulation, containerization or network level isolation to delay and deter attackers who compromise one part of an application from pivoting elsewhere in the application.	3

V15.3 Defensive Coding

This section covers vulnerability types, including type juggling, prototype pollution, and others, which result from using insecure coding patterns in a particular language. Some may not be relevant to all languages, whereas others will have language-specific fixes or may relate to how a particular language or framework handles a feature such as HTTP parameters. It also considers the risk of not cryptographically validating application updates.

It also considers the risks associated with using objects to represent data items and accepting and returning these via external APIs. In this case, the application must ensure that data fields that should not be writable are not modified by user input (mass assignment) and that the API is selective about what data fields get returned. Where field access depends on a user’s permissions, this should be considered in the context of the field-level access control requirement in the Authorization chapter.

#	Description	Level
15.3.1	Verify that the application only returns the required subset of fields from a data object. For example, it should not return an entire data object, as some individual fields should not be accessible to users.	1
15.3.2	Verify that where the application backend makes calls to external URLs, it is configured to not follow redirects unless it is intended functionality.	2
15.3.3	Verify that the application has countermeasures to protect against mass assignment attacks by limiting allowed fields per controller and action, e.g., it is not possible to insert or update a field value when it was not intended to be part of that action.	2
15.3.4	Verify that all proxying and middleware components transfer the user's original IP address correctly using trusted data fields that cannot be manipulated by the end user, and the application and web server use this correct value for logging and security decisions such as rate limiting, taking into account that even the original IP address may not be reliable due to dynamic IPs, VPNs, or corporate firewalls.	2
15.3.5	Verify that the application explicitly ensures that variables are of the correct type and performs strict equality and comparator operations. This is to avoid type juggling or type confusion vulnerabilities caused by the application code making an assumption about a variable type.	2
15.3.6	Verify that JavaScript code is written in a way that prevents prototype pollution, for example, by using Set() or Map() instead of object literals.	2
15.3.7	Verify that the application has defenses against HTTP parameter pollution attacks, particularly if the application framework makes no distinction about the source of request parameters (query string, body parameters, cookies, or header fields).	2

V15.4 Safe Concurrency

Concurrency issues such as race conditions, time-of-check to time-of-use (TOCTOU) vulnerabilities, deadlocks, livelocks, thread starvation, and improper synchronization can lead to unpredictable behavior and security risks. This section includes various techniques and strategies to help mitigate these risks.

#	Description	Level
15.4.1	Verify that shared objects in multi-threaded code (such as caches, files, or in-memory objects accessed by multiple threads) are accessed safely by using thread-safe types and synchronization mechanisms like locks or semaphores to avoid race conditions and data corruption.	3
15.4.2	Verify that checks on a resource's state, such as its existence or permissions, and the actions that depend on them are performed as a single atomic operation to prevent time-of-check to time-of-use (TOCTOU) race conditions. For example, checking if a file exists before opening it, or verifying a user's access before granting it.	3
15.4.3	Verify that locks are used consistently to avoid threads getting stuck, whether by waiting on each other or retrying endlessly, and that locking logic stays within the code responsible for managing the resource to ensure locks cannot be inadvertently or maliciously modified by external classes or code.	3
15.4.4	Verify that resource allocation policies prevent thread starvation by ensuring fair access to resources, such as by leveraging thread pools, allowing lower-priority threads to proceed within a reasonable timeframe.	3

References

For more information, see also:

- OWASP Prototype Pollution Prevention Cheat Sheet
- OWASP Mass Assignment Prevention Cheat Sheet
- OWASP CycloneDX Bill of Materials Specification
- OWASP Web Security Testing Guide: Testing for HTTP Parameter Pollution

V16 Security Logging and Error Handling

Control Objective

Security logs are distinct from error or performance logs and are used to record security-relevant events such as authentication decisions, access control decisions, and attempts to bypass security controls, such as input validation or business logic validation. Their purpose is to support detection, response, and investigation by providing high-signal, structured data for analysis tools like SIEMs.

Logs should not include sensitive personal data unless legally required, and any logged data must be protected as a high-value asset. Logging must not compromise privacy or system security. Applications must also fail securely, avoiding unnecessary disclosure or disruption.

For detailed implementation guidance, refer to the OWASP Cheat Sheets in the references section.

V16.1 Security Logging Documentation

This section ensures a clear and complete inventory of logging across the application stack. This is essential for effective security monitoring, incident response, and compliance.

#	Description	Level
16.1.1	Verify that an inventory exists documenting the logging performed at each layer of the application's technology stack, what events are being logged, log formats, where that logging is stored, how it is used, how access to it is controlled, and for how long logs are kept.	2

V16.2 General Logging

This section provides requirements to ensure that security logs are consistently structured and contain the expected metadata. The goal is to make logs machine-readable and analyzable across distributed systems and tools.

Naturally, security events often involve sensitive data. If such data is logged without consideration, the logs themselves become classified and therefore subject to encryption requirements, stricter retention policies, and potential disclosure during audits.

Therefore, it is critical to log only what is necessary and to treat log data with the same care as other sensitive assets.

The requirements below establish foundational requirements for logging metadata, synchronization, format, and control.

#	Description	Level
16.2.1	Verify that each log entry includes necessary metadata (such as when, where, who, what) that would allow for a detailed investigation of the timeline when an event happens.	2
16.2.2	Verify that time sources for all logging components are synchronized, and that timestamps in security event metadata use UTC or include an explicit time zone offset. UTC is recommended to ensure consistency across distributed systems and to prevent confusion during daylight saving time transitions.	2
16.2.3	Verify that the application only stores or broadcasts logs to the files and services that are documented in the log inventory.	2